

期末大作业实验报告

基于多人人脸检测与表情识别的课堂状态分析系统

项目	内容
课程	物联网应用软件开发
作业类型	期末综合大作业
姓名	朱业航
学号	231880098
项目名称	基于多人人脸检测与表情识别的课堂状态分析系统
开发语言	Python
主要技术	OpenCV、MediaPipe、TensorFlow/Keras、Streamlit、Pandas
报告日期	2026 年 6 月 15 日

一、项目概述

本项目实现了一个面向课堂场景的多人人脸检测与表情识别系统。系统可以接收图片、视频和摄像头画面作为输入，自动检测画面中的多人脸区域，对每张人脸进行表情识别，然后统计整体表情分布，并根据表情比例判断课堂整体状态。

系统的核心目标不是只完成单张图片上的表情分类，而是围绕“课堂状态分析”构建一个完整应用：包括多人脸检测、表情识别、课堂状态判断、视频时序分析、结果可视化、历史记录保存和结果包导出。为了方便课程演示和助教提问，本项目还对原始实现进行了模块化重构，将算法流程、时序分析和界面展示分离到不同模块中。

系统支持的输入方式包括：

- 图片上传：上传包含多个人脸的图片，输出标注结果和统计信息。
- 视频上传：按时间间隔采样视频帧，输出时序趋势、预警结果和关键采样帧。
- 摄像头拍照：通过浏览器拍照后进行单帧分析。
- 本地实时摄像头：通过 OpenCV 直接读取本机摄像头，适合课堂演示。
- WebRTC 实时流：保留为实验性入口，用于浏览器实时视频流分析。

二、任务要求与完成情况

根据期末大作业要求，需要提交实验报告和实验代码。其中实验报告需要包含每个模块的具体设计、运行结果和简述流程，至少 8 页；实验代码需要包含 `results` 文件夹，保存运行结果截图或结果文件。

本项目完成情况如下：

要求	完成情况	对应文件
多人脸检测	已实现 MediaPipe 与 Haar Cascade 双检测器合并，并使用 NMS 去重	<code>face_detector.py</code>
表情识别	已复用实验二 CNN3 模型，识别 7 类表情	<code>expression_recognizer.py</code>
课堂状态判断	已根据多人表情比例进行课堂状态规则判断	<code>analyzer.py</code>
图片上传分析	已实现图片上传、检测、识别、统计和结果图展示	<code>app.py</code> 、 <code>pipeline.py</code>
视频上传分析	已实现按秒采样、逐帧分析、趋势图、关键帧和预警	<code>temporal_analyzer.py</code>

要求	完成情况	对应文件
摄像头分析	已实现拍照模式、本地实时模式和 WebRTC 实验模式	app.py
结果保存	已实现历史 CSV、图片/摄像头结果包、视频结果包导出	utils.py
运行结果	已保存在 ClassroomStateAnalyzer/results/ 目录下	results/

三、开发环境

项目	内容
操作系统	Windows 11
Python 环境	Conda 环境 hand
Python 版本	3.11
Web 框架	Streamlit
图像处理	OpenCV、Pillow
人脸检测	MediaPipe Face Detection、OpenCV Haar Cascade
深度学习框架	TensorFlow/Keras
数据处理	Pandas、NumPy
模型文件	models/cnn3_best_weights.h5

运行方式：

```
cd ClassroomStateAnalyzer
streamlit run app.py
```

启动后浏览器访问：

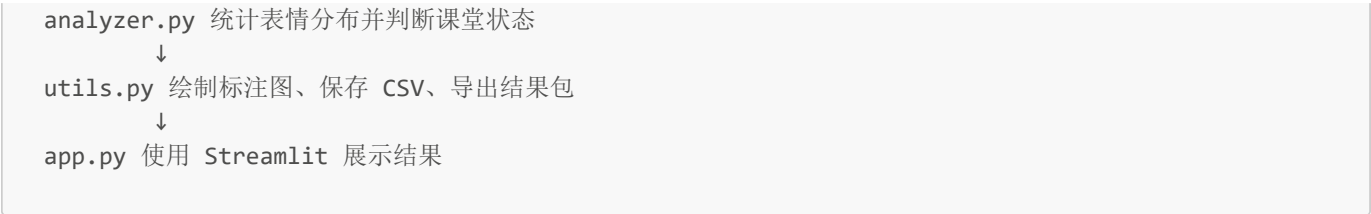
```
http://localhost:8501
```

四、系统总体设计

4.1 总体流程

系统整体流程如下：





如果输入是视频，则会增加一层时序分析：



4.2 模块划分

模块	文件	主要职责
界面调度层	app.py	Streamlit 页面、输入选择、参数设置、结果展示
人脸检测层	face_detector.py	检测图片中的多个人脸
表情识别层	expression_recognizer.py	加载 CNN3 模型并识别 7 类表情
统计规则层	analyzer.py	计算表情比例、判断课堂状态、视频预警
单帧管线层	pipeline.py	统一图片、视频帧、摄像头帧的处理流程
视频时序层	temporal_analyzer.py	视频采样、逐帧日志、趋势分析和预警
工具函数层	utils.py	中文标注、CSV 读写、结果图和结果包导出

这样的设计使系统逻辑更加清楚。图片、视频和摄像头虽然输入形式不同，但进入算法后都被统一看作一帧图像，因此都可以复用同一个 pipeline.py。

五、人脸检测模块设计

5.1 设计目标

课堂场景中的人脸检测具有以下难点：

- 人数较多，同一张图片中可能出现多个学生。
- 人脸尺度差异较大，前排和后排人脸大小不同。
- 可能存在低头、侧脸、模糊、遮挡等情况。
- 单一检测器容易出现漏检或误检。

因此，本项目没有只依赖一个检测器，而是采用 MediaPipe 与 Haar Cascade 双检测器结合的方式。

5.2 MediaPipe 检测

MediaPipe Face Detection 是一个轻量级人脸检测方案，适合实时应用。本项目使用它作为主要检测器，优点是速度快、精度较高、对常见正脸场景比较稳定。

核心步骤：

- 1. 将 OpenCV 的 BGR 图像转换为 RGB。
- 2. 调用 MediaPipe Face Detection。
- 3. 将归一化边界框转换为图像像素坐标。
- 4. 输出格式为 (x1, y1, x2, y2, confidence)。

5.3 Haar Cascade 检测

OpenCV Haar Cascade 是传统人脸检测方法。本项目将它作为补充检测器，主要用于提高召回率。虽然 Haar Cascade 的精度不如深度学习检测器，但在某些场景下可以补充 MediaPipe 漏掉的人脸。

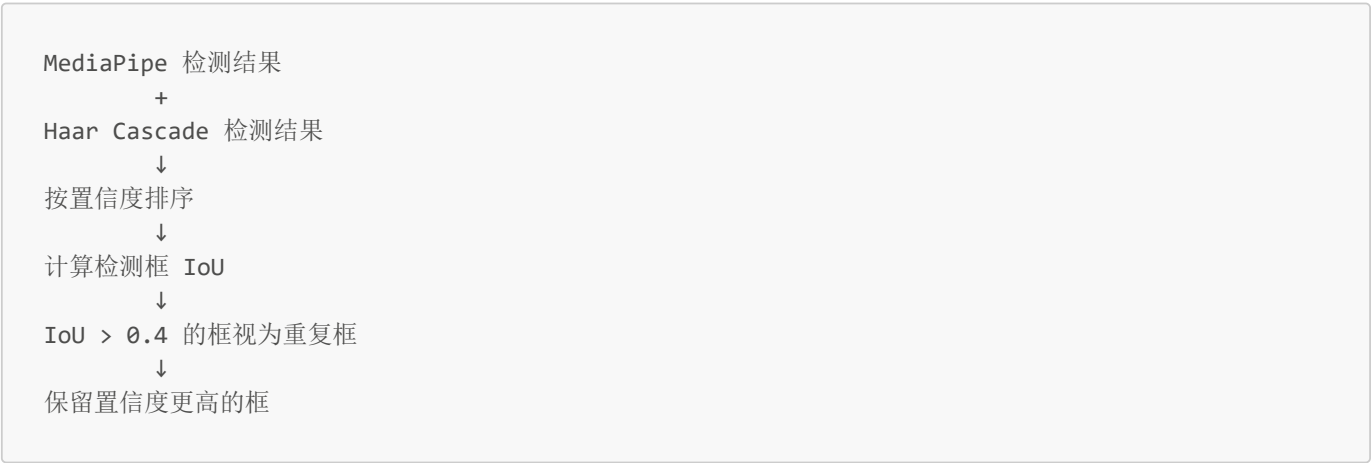
核心参数包括：

参数	作用
scaleFactor=1.1	多尺度检测时图像缩放比例
minNeighbors=3	候选框保留条件
minSize	最小人脸尺寸

5.4 双检测器合并与 NMS 去重

如果直接把两个检测器的结果简单相加，同一个人脸可能会被重复统计。因此系统使用 NMS，即非极大值抑制，对重复框进行去重。

处理流程：



这种设计提高了多人场景下的召回率，同时避免重复框导致人数统计错误。

六、表情识别模块设计

6.1 模型来源

表情识别模块复用了实验二中训练得到的 CNN3 模型，模型权重文件为：

```
ClassroomStateAnalyzer/models/cnn3_best_weights.h5
```

模型输入为 48×48×1 的灰度人脸图像，输出 7 类表情的概率分布。

7 类表情包括：

英文标签	中文含义
Angry	生气
Disgust	厌恶
Fear	恐惧
Happy	开心
Sad	悲伤
Surprise	惊讶
Neutral	中性

6.2 人脸预处理

识别前需要先对检测到的人脸区域进行预处理：

- 1. 根据人脸框从原图中裁剪人脸。
- 2. 转换为灰度图。
- 3. resize 到 48×48。
- 4. 像素值归一化到 [0, 1]。
- 5. 增加 batch 维度，送入 CNN3 模型。

6.3 普通模式与快速模式

项目中提供两种识别策略：

模式	方法	适用场景
普通模式	对每个人脸生成多个增强版本，再取平均结果	图片分析，追求稳定性
快速模式	多个人脸组成 batch，一次模型推理	视频和摄像头实时分析

快速模式的意义在于降低延迟。视频和实时摄像头需要连续处理多帧，如果每个人脸都进行多次增强推理，会明显降低响应速度。因此系统默认在实时场景中启用快速模式。

6.4 模型单例与预热

为了避免每次分析都重新加载模型，系统使用单例模式管理表情识别器：

```
recognizer = get_recognizer()
recognizer.warm_up()
```

warm_up() 会在系统启动时执行一次空预测，减少 TensorFlow 第一次推理时的冷启动延迟。

七、统一单帧分析管线设计

7.1 设计原因

图片、视频和摄像头本质上都可以转化为图像帧。如果每一种输入模式都单独写一套检测、识别、统计和绘图逻辑，容易产生以下问题：

- 参数不一致，例如图片模式和视频模式使用不同置信度阈值。
- 代码重复，后续修改成本高。

- 演示时不容易解释系统结构。

因此系统新增 `pipeline.py`，将单帧分析过程统一封装为：

```
analyze_frame(frame_bgr, recognizer, config)
```

7.2 AnalysisConfig

`AnalysisConfig` 用于集中管理核心分析参数：

参数	含义
<code>fast_mode</code>	是否启用快速识别模式
<code>confidence_threshold</code>	表情识别置信度阈值
<code>detection_sensitivity</code>	人脸检测灵敏度
<code>merge_detectors</code>	是否启用双检测器合并

7.3 单帧处理步骤

`analyze_frame()` 的内部流程为：

1. 调用 `detect_faces()` 检测人脸框。
2. 调用表情识别器识别每个人脸的表情。
3. 根据置信度阈值过滤低置信度结果。
4. 调用 `calculate_statistics()` 统计表情分布。
5. 调用 `judge_classroom_state()` 判断课堂状态。
6. 调用 `draw_results()` 绘制人脸框和中文表情标签。
7. 返回分析结果和耗时。

返回结果包括：

字段	说明
<code>faces</code>	检测到的人脸框
<code>raw_recognition_results</code>	过滤前的识别结果
<code>recognition_results</code>	过滤后的识别结果
<code>stats</code>	表情统计信息
<code>state_text</code>	课堂状态文字
<code>state_level</code>	课堂状态等级
<code>result_image</code>	标注后图像
<code>elapsed_ms</code>	单帧分析耗时

八、统计分析与课堂状态判断

8.1 表情统计

`analyzer.py` 中的 `calculate_statistics()` 会对所有识别结果进行汇总，输出：

- 检测人数。

- 每种表情的人数。
- 每种表情的比例。
- 当前主要表情。

例如，如果检测到 5 个人，其中 3 个为 Neutral，2 个为 Sad，则统计结果中 `Neutral_ratio=0.6`，`Sad_ratio=0.4`。

8.2 课堂状态规则

课堂状态判断采用可解释的规则方法。规则如下：

优先级	判断条件	课堂状态
1	检测人数为 0	未检测到学生
2	Happy + Neutral 占比 $\geq 70\%$	课堂状态良好
3	Sad + Angry 占比 $\geq 40\%$	课堂状态较低落，需要关注
4	Neutral 是主要表情	课堂状态平稳
5	Surprise 占比 $\geq 30\%$	课堂注意力波动较大
6	其他情况	课堂状态一般

使用规则判断的优点是逻辑清晰、可解释性强，适合课程项目演示。助教或教师可以直接根据表情比例理解系统为什么给出某种课堂状态。

8.3 视频预警规则

为了让视频分析不仅是单帧统计的简单叠加，系统增加了视频时序预警：

预警等级	触发条件
绿色状态	连续 10 个采样帧课堂状态良好
黄色预警	连续 5 个采样帧中性比例偏高
红色预警	连续 3 个采样帧处于低落或注意力波动状态

预警结果会在视频分析报告中展示，并保存到 `video_summary.json` 中。

九、视频时序分析模块设计

9.1 设计动机

视频和图片最大的区别在于视频具有时间维度。如果只把视频中的所有采样帧识别结果合并后统计，就会出现一个问题：同一个学生不同帧中会被重复统计为多个人。

因此，系统对视频统计口径进行了重新设计：

- 不把跨帧累计人脸数称为“检测人数”。
- 使用“人脸实例数”表示所有采样帧中的累计人脸出现次数。
- 使用“平均每帧人数”表示课堂画面中的平均人数。
- 使用“峰值人数”表示某个采样帧中最多检测到的人数。
- 表情比例使用每个采样帧比例的平均值，而不是简单累计所有人脸后计算。

9.2 视频分析流程

`temporal_analyzer.py` 的主要函数为：

```
analyze_video(video_path, recognizer, config, sample_every_seconds, max_frames)
```

处理流程：



9.3 frame_records 字段

视频逐帧日志中每条记录包括：

字段	含义
sample_index	第几个采样帧
frame_index	原视频帧号
time_sec	视频时间点
total_count	当前采样帧检测人数
main_expression	当前采样帧主要表情
state_text	当前采样帧课堂状态
elapsed_ms	当前采样帧分析耗时
Happy_ratio 等	各表情在当前帧中的比例

十、结果保存与导出设计

10.1 历史记录

系统使用 records.csv 保存轻量历史记录。每次完成一次图片、视频或摄像头分析后，会保存一条摘要记录，字段包括：

- 时间。
- 输入文件名。
- 检测人数或视频平均人数。
- 各表情数量或比例。
- 主要表情。
- 课堂状态。

历史记录主要用于界面中的历史表格和趋势图。

10.2 当前结果包

为了满足“代码文件中包含 results 文件夹，包含运行结果截图”的提交要求，系统重新设计了结果保存逻辑。现在保存结果不是只保存单张图片，而是根据输入模态保存完整结果包。

图片、拍照和本地实时摄像头结果包包括：

```
annotated_result.jpg
summary.json
face_details.csv
```

视频结果包包括：

```
video_summary.json
temporal_records.csv
01_开始帧.jpg
02_中间帧.jpg
03_结束帧.jpg
04_人数峰值帧.jpg
```

这种设计可以避免把视频结果错误地展示成单张图片结果，同时也让实验代码提交时的 results 文件夹更有证明价值。

十一、运行结果展示

本章按照“图片单帧分析 → 视频时序分析 → 本地实时摄像头分析”的顺序展示运行结果。这样可以体现系统能力由浅入深：先说明基础检测识别功能可以稳定运行，再展示视频模态中的时序分析和预警，最后展示系统可以接入摄像头完成实时演示。

11.1 图片上传模式结果

图片上传模式是系统最基础的功能，用于验证“多人脸检测 → 表情识别 → 课堂状态判断 → 标注图导出”这一条完整单帧处理链路。

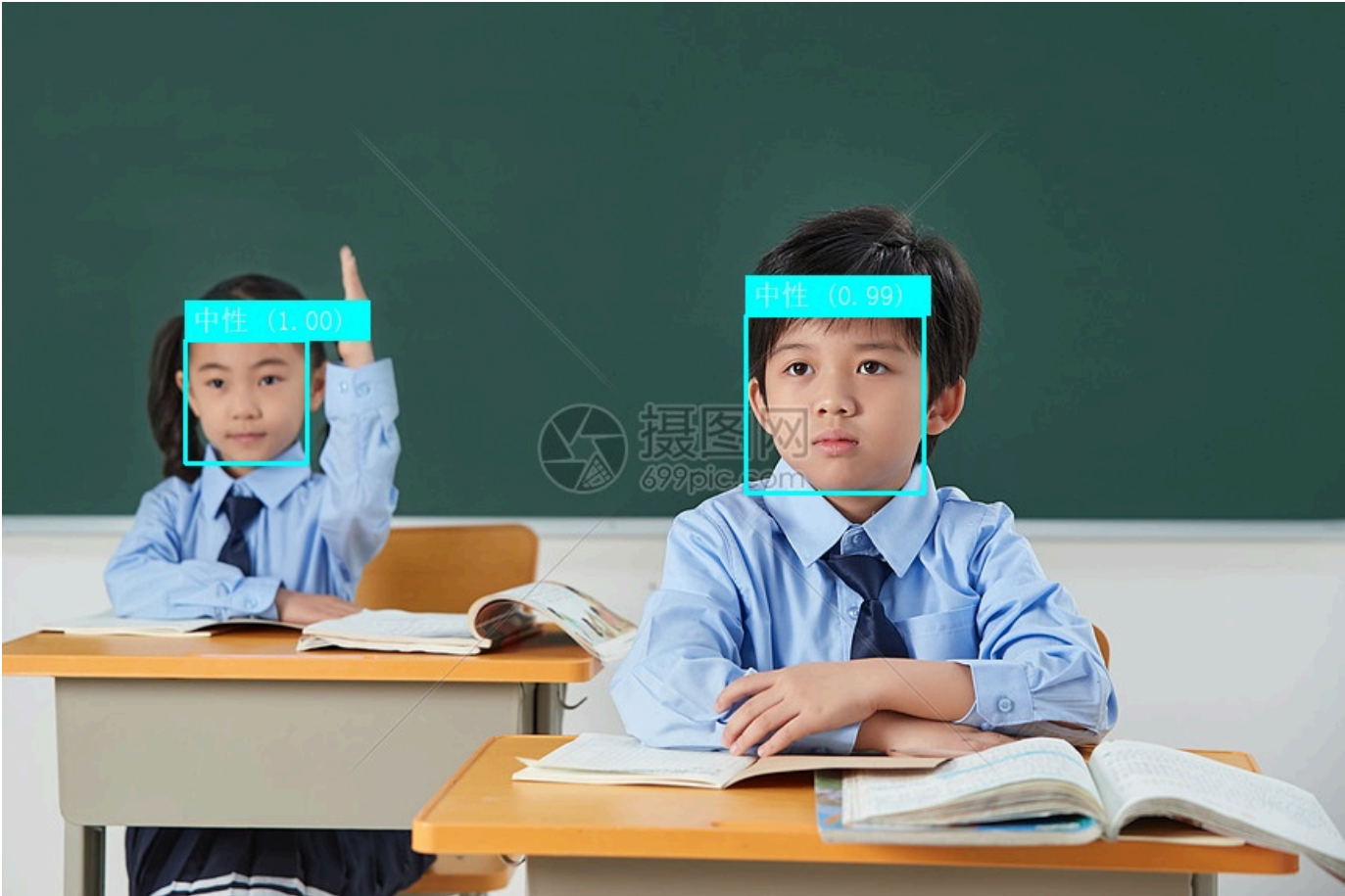
示例图片运行结果保存在：

```
ClassroomStateAnalyzer/results/1_result_20260615_153615.jpg
```

该图片结果来自一次图片上传检测，历史记录中对应输入为 1.jpg，检测到 2 人，主要表情为 Neutral，课堂状态判断为“课堂状态良好”。

指标	结果
输入文件	1.jpg
检测人数	2
Happy	0
Neutral	2
Sad	0
主要表情	Neutral（中性）
课堂状态	课堂状态良好

图片标注结果如下：



通过该结果可以看到，系统已经能够在静态图片中完成人脸定位、表情标签绘制和课堂状态判断。图片模式是后续视频分析和摄像头分析的基础，因为视频帧和摄像头帧最终也会进入同一个 `pipeline.analyze_frame()` 单帧分析流程。

11.2 视频分析结果

在图片模式验证通过后，进一步测试视频上传模式。视频模式不再只展示某一张图片的前后对比，而是按采样帧生成视频时序报告。

视频结果包路径：

```
ClassroomStateAnalyzer/results/video_1_20260615_162916/
```

该结果包包含：

文件	说明
video_summary.json	视频整体分析摘要
temporal_records.csv	逐帧时序分析日志
01_开始帧.jpg	开始采样帧
02_中间帧.jpg	中间采样帧
03_结束帧.jpg	结束采样帧
04_人数峰值帧.jpg	人数峰值采样帧

视频摘要结果：

指标	数值
视频文件	1.mp4
采样帧数	20
人脸实例数	24
平均每帧人数	1.2
峰值人数	2
主要表情	Neutral（中性）
Happy 平均比例	2.5%
Neutral 平均比例	60.0%
Sad 平均比例	27.5%
课堂状态	课堂状态平稳
预警结果	红色预警
预警触发帧	第 6 个采样帧

性能结果：

指标	数值
视频总帧数	2213
FPS	30.0
采样间隔	每 30 帧采样 1 帧
采样间隔秒数	1.0 秒
分析采样帧数	20
总耗时	6.42 秒
平均每帧耗时	320.86 ms

视频关键采样帧如下。

开始帧：



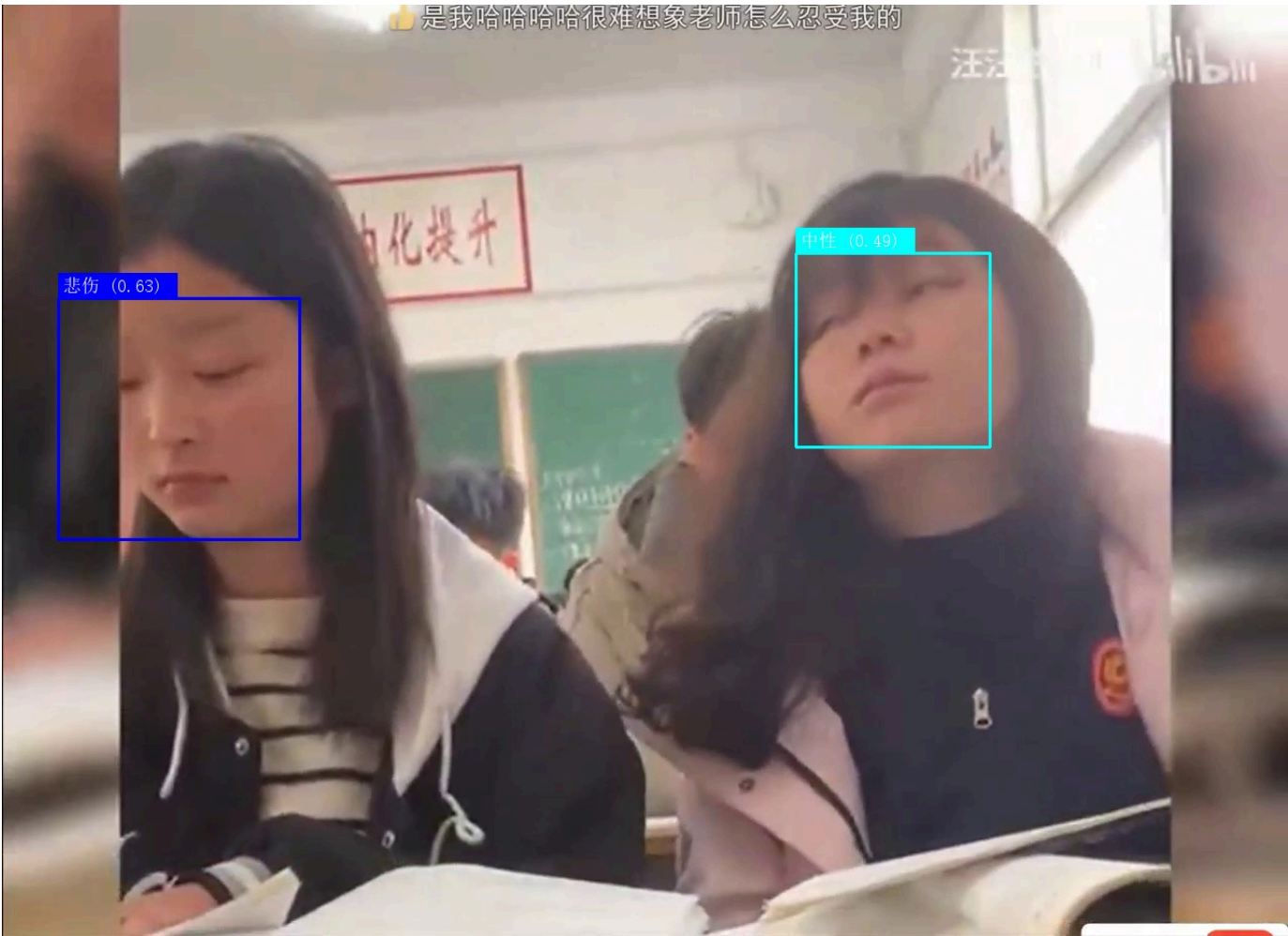
中间帧：



结束帧：



人数峰值帧：



11.3 视频逐帧日志示例

temporal_records.csv 中的部分记录如下：

sample_index	time_sec	total_count	main_expression	state_level	Happy_ratio	Neutral_ratio	Sad_ratio
1	0.0	0	Neutral	empty	0.0	0.0	0.0
2	1.0	0	Neutral	empty	0.0	0.0	0.0
3	2.0	1	Neutral	good	0.0	1.0	0.0
4	3.0	2	Neutral	low	0.0	0.5	0.5
5	4.0	2	Neutral	low	0.0	0.5	0.5
6	5.0	2	Neutral	low	0.0	0.5	0.5
7	6.0	2	Neutral	low	0.0	0.5	0.5
8	7.0	1	Neutral	good	0.0	1.0	0.0
9	8.0	1	Neutral	good	0.0	1.0	0.0
10	9.0	1	Neutral	good	0.0	1.0	0.0

从表中可以看到，第 4 到第 6 个采样帧连续出现低落状态，因此触发红色预警。这说明视频分析不仅保存整体结果，也能分析课堂状态随时间的变化。

11.4 本地实时摄像头结果

在图片和视频分析功能都验证完成后，最后测试本地实时摄像头模式。该模式通过 OpenCV 直接读取本机摄像头，不依赖 WebRTC 连接协商，因此更适合现场演示。

本地实时摄像头结果包路径：

ClassroomStateAnalyzer/results/camera_local_camera_20260615_164213_20260615_164222/

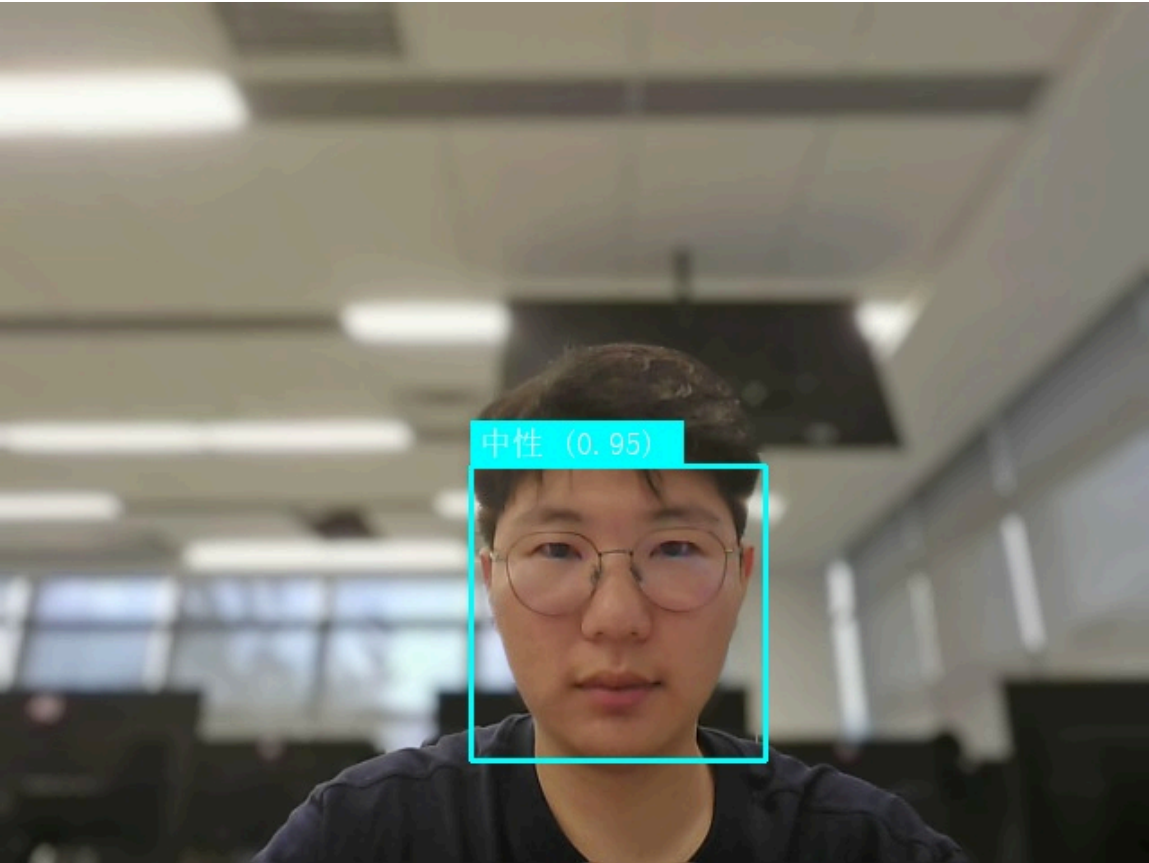
该结果包包含：

文件	说明
annotated_result.jpg	摄像头最后一次分析的标注图
summary.json	单帧统计摘要
face_details.csv	单个人脸识别明细

本次摄像头结果摘要：

指标	结果
输入来源	local_camera_20260615_164213.jpg
检测人数	1
主要表情	Neutral（中性）
Neutral 比例	100.0%
课堂状态	课堂状态良好

摄像头标注结果如下：



11.5 历史记录示例

历史记录保存在：

```
ClassroomStateAnalyzer/records.csv
```

其中部分记录如下：

时间	图片名称	检测人数	Happy	Neutral	Sad	主要表情	课堂状态
2026-05-28 19:35:21	12019-afghanistan-80087_1920.jpg	9	0	7	2	Neutral	课堂状态良好
2026-05-28 19:40:29	publicdomainpictures-classroom-15593_1920.jpg	2	1	1	0	Happy	课堂状态良好
2026-05-28 19:40:43	van-tay-media-auRYODyMG0E-unsplash.jpg	5	1	0	3	Sad	课堂状态较低落
2026-06-11 11:49:21	2.jpg	3	0	3	0	Neutral	课堂状态良好
2026-06-11 11:50:12	2b5990b0-8579-4bf2-835f-5a6779288f7f.png	5	1	0	2	Sad	课堂状态较低落

十二、测试与验证

12.1 静态检查

开发过程中对核心模块进行了静态编译检查：

```
python -m py_compile ClassroomStateAnalyzer/app.py \
ClassroomStateAnalyzer/pipeline.py \
ClassroomStateAnalyzer/temporal_analyzer.py \
ClassroomStateAnalyzer/analyzer.py \
ClassroomStateAnalyzer/utils.py
```

检查结果无 Python 语法错误。

12.2 图片模式验证

图片模式验证内容：

- 上传多人图片。
- 点击开始检测。
- 检查原图和标注图是否正常显示。
- 检查检测人数、主要表情、表情比例和课堂状态是否正常显示。
- 检查历史记录是否写入 records.csv。
- 检查结果包是否能保存。

12.3 视频模式验证

视频模式验证内容：

- 上传 `1.mp4`。
- 设置采样间隔为 1 秒。
- 最大采样帧数设置为 20。
- 检查是否生成 20 条逐帧日志。
- 检查是否显示平均每帧人数、人脸实例数和峰值人数。
- 检查是否显示关键采样帧。
- 检查是否生成 `video_summary.json` 和 `temporal_records.csv`。

12.4 摄像头模式验证

摄像头模式验证内容：

- 拍照模式可以正常调用浏览器摄像头并分析。
- 本地实时模式可以通过 OpenCV 读取本机摄像头。
- WebRTC 模式保留为实验模式，如果出现连接较慢，可以使用本地实时模式完成课堂演示。

十三、遇到的问题与优化

13.1 WebRTC 实时流连接问题

在浏览器实时流模式中，曾出现：

```
Connection is taking longer than expected
```

该问题与 WebRTC 的 ICE/STUN 协商、浏览器权限和网络环境有关。为提高演示稳定性，系统增加了 OpenCV 本地实时摄像头模式，不依赖浏览器 WebRTC 协商，适合本机课堂演示。

13.2 视频统计口径问题

原始视频统计容易把同一个人在不同采样帧中的出现累计为多个人。为解决该问题，系统将视频统计改为：

- 人脸实例数：跨帧累计的人脸出现次数。
- 平均每帧人数：更适合表示视频中的课堂规模。
- 峰值人数：单帧最大检测人数。
- 按帧平均表情比例：避免长视频或多人帧对结果产生不合理影响。

13.3 保存结果逻辑问题

原本“保存结果图片”的逻辑只适合图片，不适合视频。现在系统改为“保存当前结果包”：

- 图片和摄像头保存标注图、摘要和人脸明细。
- 视频保存摘要、时序 CSV 和关键帧。

这样提交代码时，`results` 文件夹中能够完整反映系统运行结果。

13.4 模块化优化

原先部分逻辑集中在 `app.py` 中，不利于讲解和维护。优化后：

- `app.py` 只负责界面。
- `pipeline.py` 负责单帧处理。
- `temporal_analyzer.py` 负责视频时序。
- `analyzer.py` 负责统计规则。
- `utils.py` 负责保存和导出。

这种结构更符合软件工程中的分层设计思想。

十四、创新点与特色实现

本项目的特色实现包括：

- 1. **双检测器合并**：MediaPipe 与 Haar Cascade 并行检测，再通过 NMS 去重，提高多人场景下召回率。
- 2. **统一单帧 pipeline**：图片、视频帧和摄像头帧共享同一分析流程，减少重复代码。
- 3. **视频时序分析**：不是简单统计所有帧，而是输出逐帧日志、趋势变化和三级预警。
- 4. **视频统计口径优化**：区分人脸实例数、平均每帧人数和峰值人数。
- 5. **本地实时摄像头模式**：解决 WebRTC 在本机演示中不稳定的问题。
- 6. **按模式导出结果包**：图片/摄像头和视频分别保存不同结构的运行结果。
- 7. **中文可视化标注**：通过 PIL 绘制中文表情标签，解决 OpenCV 中文显示问题。

十五、局限性与改进方向

虽然系统已经完成主要功能，但仍有一些局限：

局限性	改进方向
表情识别模型基于 FER2013，真实课堂适应性有限	使用更大规模数据集或课堂场景数据微调模型
视频未做跨帧身份跟踪	引入目标跟踪或人脸 ReID，统计唯一学生人数
课堂状态规则较简单	引入机器学习模型或多模态特征进行状态判断
WebRTC 受浏览器和网络影响	演示时优先使用本地实时模式，部署时配置 HTTPS/TURN
人脸遮挡和侧脸仍可能漏检	引入更强的人脸检测模型，如 RetinaFace 或 YOLO-face

十六、总结

本项目完成了一个完整的课堂状态分析系统，能够对图片、视频和摄像头画面进行多人人脸检测与表情识别，并根据表情分布判断课堂状态。系统不仅完成了基础识别功能，还增加了视频时序分析、三级预警、本地实时摄像头、结果包导出等功能。

从软件结构上看，项目采用模块化设计，将人脸检测、表情识别、统计规则、单帧管线、视频时序和界面展示分离，使代码结构更加清晰，也便于向助教说明每个部分的实现位置和方法。

从运行结果看，系统已经生成了图片/摄像头结果包、视频结果包和历史记录 CSV，`results` 文件夹可以直接作为实验代码提交时的运行结果证明。整体上，本项目满足期末大作业对实验报告、代码和运行结果的提交要求。

附录：核心文件说明

文件	作用
<code>ClassroomStateAnalyzer/app.py</code>	Streamlit 主程序入口
<code>ClassroomStateAnalyzer/face_detector.py</code>	多人人脸检测
<code>ClassroomStateAnalyzer/expression_recognizer.py</code>	表情识别模型加载与推理
<code>ClassroomStateAnalyzer/analyzer.py</code>	表情统计、课堂状态判断和视频预警
<code>ClassroomStateAnalyzer/pipeline.py</code>	单帧统一分析流程

文件	作用
ClassroomStateAnalyzer/temporal_analyzer.py	视频时序分析
ClassroomStateAnalyzer/utils.py	标注绘制、CSV 和结果包导出
ClassroomStateAnalyzer/results/	运行结果文件夹
ClassroomStateAnalyzer/records.csv	历史检测记录